

Instrucciones para configurar y ejecutar el proyecto

Requisitos

- Docker y Docker Compose. En Windows y macOS vienen incluidos en Docker Desktop.
- Los puertos 5432 (PostgreSQL), 8080 (Airflow) y 8000 (API) libres.

No hace falta instalar Python ni PostgreSQL en el equipo: todo corre dentro de contenedores.

Contenido del repositorio

```
docker-compose.yml   Define los tres servicios: PostgreSQL, Airflow y la API
passwords.json       Contraseña del usuario admin de Airflow
sql/erco.sql         Crea los usuarios, la base de datos, las tablas y las vistas
dags/                Un DAG de Airflow por tabla
data/                Archivos CSV de origen
api/                 Código de la API (FastAPI) y su Dockerfile
```

Paso 1. Obtener el código

```
git clone https://github.com/ksantyr/prueba_tecnica_erco.git
cd prueba_tecnica_erco
```

Todos los comandos siguientes se ejecutan desde esa carpeta.

Paso 2. Levantar los servicios

```
docker compose up -d --build
```

Crea y arranca los tres contenedores: `postgresql`, `airflow` y `api`. La primera vez descarga las imágenes y puede tardar varios minutos. Para revisar que estén en ejecución:

```
docker compose ps
```

Después de que el contenedor de Airflow aparece en ejecución, Airflow necesita uno o dos minutos más para terminar de iniciar.

Paso 3. Crear la base de datos

```
docker cp sql/erco.sql postgresql:/tmp/erco.sql
docker exec postgresql psql -U postgres -d postgres -v ON_ERROR_STOP=1
-f /tmp/erco.sql
```

El script crea los usuarios `erco_escritura` y `erco_lectura`, la base `erco` y las seis tablas con sus índices y vistas. Debe terminar sin errores, con `RESET` como última línea. Se ejecuta una sola vez: si se repite, falla porque la base ya existe (ver "Empezar desde cero").

Paso 4. Crear la conexión de Airflow a la base de datos

```
docker exec airflow airflow connections add erco_postgres --conn-uri "p
ostgresql://erco_escritura:erco_escritura@postgresql:5432/erco"
```

Los DAGs usan esta conexión para escribir en la base. Si el comando falla porque Airflow todavía está iniciando, esperar un minuto y repetirlo.

Paso 5. Cargar los datos

Hay un DAG por tabla. Se ejecutan uno por uno y en este orden, porque las llaves foráneas exigen que primero existan los servicios y luego las mediciones:

```
docker exec airflow airflow dags test erco_carga_services
docker exec airflow airflow dags test erco_carga_records
docker exec airflow airflow dags test erco_carga_consumption
docker exec airflow airflow dags test erco_carga_injection
docker exec airflow airflow dags test erco_carga_tariffs
docker exec airflow airflow dags test erco_carga_xm_data_hourly_per_age
nt
```

Cada comando ejecuta las tres tareas del DAG (extraer, transformar y cargar) y debe terminar con estado `success` antes de pasar al siguiente.

Para comprobar la carga:

```
docker exec postgresql psql -U erco_lectura -d erco -c "SELECT (SELECT
count(*) FROM vw_services) AS services, (SELECT count(*) FROM vw_record
s) AS records, (SELECT count(*) FROM vw_consumption) AS consumption, (S
ELECT count(*) FROM vw_injection) AS injection, (SELECT count(*) FROM v
```

```
w_tariffs) AS tariffs, (SELECT count(*) FROM vw_xm_data_hourly_per_agen
t) AS xm;"
```

Resultado esperado:

services	records	consumption	injection	tariffs	xm
3	2160	2160	2160	105	720

Paso 6. Activar la carga diaria (opcional)

Los DAGs quedan pausados al crearse. Para que se ejecuten todos los días a las 08:00, hora de Colombia:

1. Entrar a Airflow en http://localhost:8080 con el usuario `admin` y la contraseña del archivo `passwords.json`.
2. Activar los seis DAGs `erco_carga_*` con el interruptor de cada uno.

Al activarlos, Airflow lanza de inmediato la ejecución más reciente. Como los datos ya están cargados, no se duplican: la carga solo inserta los registros nuevos y actualiza los que cambiaron.

Paso 7. Probar la API

La forma más sencilla es la documentación interactiva: abrir http://localhost:8000/docs, elegir un endpoint, pulsar "Try it out", escribir los datos y pulsar "Execute".

Los datos de origen son de septiembre de 2023 y tienen tres servicios: 3222, 2256 y 2478. Desde una terminal de macOS o Linux también se puede probar con curl:

```
curl -X POST http://localhost:8000/calculate-invoice -H "Content-Type:
application/json" -d '{"service_id": 2256, "year": 2023, "month": 9}'
curl http://localhost:8000/client-statistics/2256
curl http://localhost:8000/system-load
```

Montos esperados de la factura de septiembre de 2023:

Servicio	EA	EE1	EE2	EC
3222	17389790.46	-201458.03	0.0	8131.85
2256	294241.59	-294241.59	225869.29	14243.58
2478	400620.71	-400620.71	174350.73	17425.45

Accesos

Servicio	Dirección	Usuario	Contraseña
API (documentación)	http://localhost:8000/docs	No requiere	No requiere
Airflow	http://localhost:8080	`admin`	La del archivo `passwords.json`

Servicio	Dirección	Usuario	Contraseña
PostgreSQL, administrador	localhost:5432	`postgres`	`postgres`
PostgreSQL, escritura (Airflow)	localhost:5432, base `erco`	`erco_escritura`	`erco_escritura`
PostgreSQL, solo lectura (API)	localhost:5432, base `erco`	`erco_lectura`	`erco_lectura`

Detener y volver a iniciar

- `docker compose stop` detiene los servicios y `docker compose start` los vuelve a iniciar, sin perder nada.
- `docker compose down` borra los contenedores. Los datos de PostgreSQL se conservan, pero Airflow guarda su historial y la conexión dentro de su contenedor, así que después de `docker compose up -d` hay que repetir el paso 4.